1 UNDERSTANDING ALGORITHMS

GENERAL VOCABULARY

construct a command to control the order/flow in which instructions are executed (e.g. sequences, selection, repetition)

It is important to be able to **construct** algorithms and be able to read them and follow their logic in solving particular problems.

LEARNING OBJECTIVES

- Understand what an algorithm is
- Understand what algorithms are used for
- Interpret algorithms as flowcharts, pseudocode and written descriptions
- Use and describe the purpose of arithmetic operators
- Understand how to code an algorithm in a high-level language

AN EXAMPLE OF AN ALGORITHM

SUBJECT VOCABULARY

unambiguous this means that the instructions cannot be misunderstood. Simply saying 'turn' would be ambiguous (i.e. unclear) because you could turn left or right. All instructions given to a computer must be unambiguous or it won't know what to do

sequence an ordered set of instructions

algorithm a precise method for solving a problem

An interactive map is a useful way to find a route between two locations. Figure 1.1 shows a route between two cities that was calculated by a mapping program.

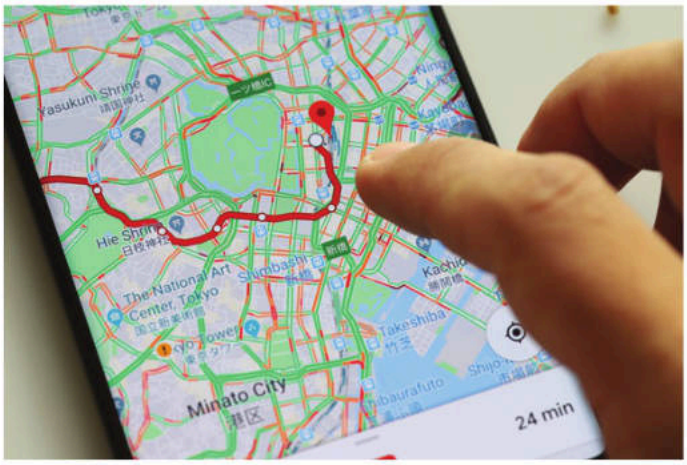
The route on this interactive map has been calculated using an algorithm.

- It is **unambiguous** in telling the driver exactly what to do, like 'turn left', 'turn right' or 'go straight'.
- It is a **sequence** of steps.
- It can be used again and will always provide the same result.
- It provides a solution to a problem, in this case, how to get from Beijing to Shanghai.

A solution to a problem with these characteristics is called an **algorithm**. Most problems have more than one solution, so different algorithms can be created for the same problem.

DID YOU KNOW?

The computer program that created the algorithm to map travel from Beijing to Shanghai was following an algorithm of its own – an algorithm ordering it how to create another algorithm!



▶ **Figure 1.1** A route calculated by a mapping program

SUCCESSFUL ALGORITHMS

There are three points to consider when deciding whether an algorithm is successful or not.

GENERAL VOCABULARY

outcome the final result of an action

consistency not changing; always the same

atlas a book of maps

- Accuracy – it must lead to the expected **outcome** (e.g. create a route from Beijing to Shanghai).
- **Consistency** – it must produce the same result each time it is run.
- Efficiency – it must solve the problem in the shortest possible time, using as few computer resources as possible. In this example, the mapping software is replacing a manual method. If it were no faster than looking in an **atlas**, then it would not be an improvement on the older method. Later in the unit there is a section on algorithms that are used to sort and search data. Some of these algorithms are more efficient than others and will sort the data far more quickly.

THE RELATIONSHIP BETWEEN ALGORITHMS AND PROGRAMS

Algorithms and programs are closely related, but they are not the same. An algorithm is a detailed design for a solution; a program is when that design is implemented.

SUBJECT VOCABULARY

high-level programming language a programming language that is similar to natural human language

This unit is all about algorithms. We look at how algorithms are implemented in **high-level programming languages** in Unit 2.

DISPLAYING AN ALGORITHM

We carry out many everyday tasks using algorithms because we are following a set of instructions to achieve an expected result, for example, making a cup of coffee. If we have performed the task many times before, we usually carry out the instructions without thinking. But if we are doing something unfamiliar, such as putting together a flat-pack chest of drawers, then we follow the instructions very carefully.

An algorithm can be expressed in different ways.

WRITTEN DESCRIPTIONS

A written description is the simplest way of expressing an algorithm. Here is an algorithm describing the everyday task of making a cup of instant coffee:



ALGORITHM FOR MAKING A CUP OF COFFEE

Fill kettle with water.
Turn on kettle.
Place coffee in cup.
Wait for water to boil.
Pour water into cup.
Add milk and sugar.
Stir.

SKILLS REASONING, PROBLEM SOLVING

ACTIVITY 1

GETTING TO SCHOOL

Produce a written description of an algorithm for getting to school. It should start with leaving home and end with arriving at school. For example, the algorithm could start with 'Walk to bus stop'.

Check your algorithm with other members of the group. Would your algorithm work for others? Are there any general statements that are common to all algorithms?

SUBJECT VOCABULARY

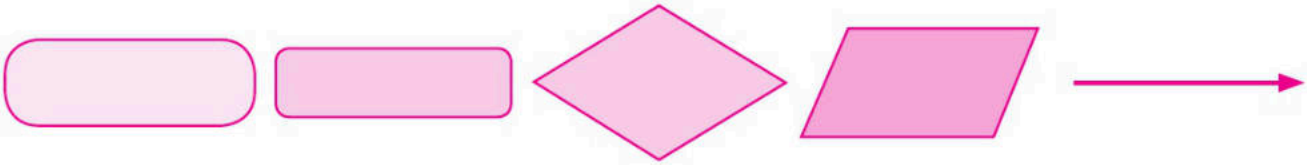
flowchart shows an algorithm as a diagram. Each step in the algorithm is represented by a symbol. Symbols are linked together with arrows showing the order in which steps are completed

FLOWCHARTS

Flowcharts can be used to show an algorithm as a diagram. They provide a more visual display.

There are special symbols that have to be used in a flowchart. You can't just make up your own, because nobody else would be able to follow your algorithm.

Figure 1.2 shows the flowchart symbols that should be used.

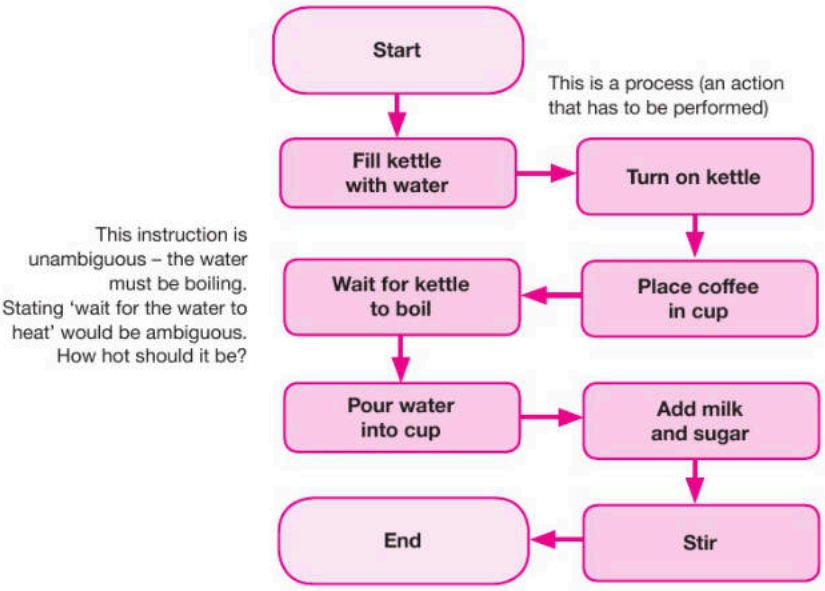


Indicates the start or end of an algorithm	Indicates a process to be carried out	Indicates a decision to be made	Indicates an input or output	Shows the logical flow of the algorithm

▲ **Figure 1.2** Flowchart symbols

The flowchart in Figure 1.3 is an alternative way of showing the algorithm for making a cup of coffee as a written description.

▶ **Figure 1.3** Flowchart of an algorithm to make a cup of coffee



SKILLS PROBLEM SOLVING

ACTIVITY 2

SCHOOL JOURNEY FLOWCHART

Display the 'journey to school' algorithm that you created in Activity 1 as a flowchart.

SKILLS PROBLEM SOLVING

ACTIVITY 3

BATH FLOWCHART

A student has created a written algorithm for preparing a bath. Working with a partner, display the following as a flowchart. You may need to change the order or add actions.

- Put in the plug.
- Fill the bath to the correct level.
- Check the temperature is OK.

GENERAL VOCABULARY

basis an important idea or fact that something is based on

SUBJECT VOCABULARY

pseudocode a structured, code-like language that can be used to describe an algorithm

developer a person whose job it is to create new software

logic the principles and reasoning underlying the constructs and elements to be applied in solving problems

The algorithms you have looked at so far are designed for humans to follow. Algorithms also form the **basis** of computer programs. Computers are mindless machines that simply do exactly what they are told. They follow a set of instructions, but they can carry out these instructions far more quickly than humans. That is why they are so useful.

PSEUDOCODE

In addition to flowcharts and written descriptions, algorithms can also be expressed in **pseudocode**. The pseudocode can be used to code the solution in an actual programming language.

It allows the **developer** to concentrate on the **logic** and efficiency of the algorithm without having to bother about the rules of any particular programming language. It is relatively straightforward to translate an algorithm written in pseudocode into any high-level programming language.

SKILLS RESEARCH

ACTIVITY 4

INVESTIGATING PSEUDOCODE

Different organisations or examination boards have their own unique versions of pseudocode. Investigate the Pearson Edexcel pseudocode that you will need for your International GCSE course and which will be used in this book.

EXAMPLE OF A SIMPLE ALGORITHM

To introduce the Pearson Edexcel pseudocode, here is a simple written algorithm that asks the user to input two numbers and then outputs the result of adding them together.

WRITTEN DESCRIPTION

ALGORITHM FOR ADDING TWO NUMBERS

Enter first number.
Enter second number.
Calculate total by adding first and second numbers.
Output total.

FLOWCHART



▲ **Figure 1.4** Flowchart showing the adding of two numbers

SUBJECT VOCABULARY

variable a 'container' used to store data. The data stored in a variable is referred to as a value. The value stored in a variable is not fixed. The same variable can store different values during the course of a program and each time a program is run

identifier a unique name given to a variable or a constant. Using **descriptive** names for variables makes code much easier to read

arithmetic operator an **operator** that performs a calculation on two numbers

GENERAL VOCABULARY

operator a character that represents an action, e.g. 'x' represents a multiplication and '/' a division

descriptive describing something clearly

PSEUDOCODE

ALGORITHM FOR ADDING TWO NUMBERS

```
SEND 'Please enter the first number' TO DISPLAY
RECEIVE firstNumber FROM KEYBOARD
SEND 'Please enter the second number' TO DISPLAY
RECEIVE secondNumber FROM KEYBOARD
SET total TO firstNumber + secondNumber
SEND total TO DISPLAY
```

The pseudocode gives clear step-by-step instructions that the computer will be expected to carry out. It also introduces some important programming concepts.

- The numbers entered by the user are stored in two **variables** with the **identifiers** `firstNumber` and `secondNumber`.
- The result of adding the numbers together is stored in the variable `total`.
- Text has to be placed in quotation marks (single or double) if it is to be displayed. For example, 'Please enter the first number' (or "Please enter the first number").
- Quotation marks are not used if a variable is to be displayed. If they were, the word 'total' in the last instruction would be displayed instead of the number it represents.
- **Arithmetic operators** are used to perform calculations. Table 1.1 shows the arithmetic operators.

ARITHMETIC OPERATORS

OPERATOR	FUNCTION	EXAMPLE
+	Addition: add the values together.	8 + 5 = 13 myScore1 + myScore2
–	Subtraction: subtract the second value from the first.	17 – 4 = 13 myScore1 – myScore2
*	Multiplication: multiply the values together.	6 * 9 = 54 numberBought * price
/	Real division: divide the first value by the second value and return the result including decimal places.	13 / 4 = 3.25 totalMarks/numberTests
DIV	**Quotient**: like division, but it only returns the whole number or *integer*.	13 DIV 4 = 3 totalMarks DIV numberTests
MOD	**Modulus**/modulo: this will return the remainder of a division.	13 / 4 = 3 remainder 1 Therefore 13 MOD4 = 1
^	Exponentiation: this is for 'to the **power** of'.	3 ^ 3 = 27 It is the same as writing 3^3

▲ **Table 1.1** Arithmetic operators

VARIABLES AND CONSTANTS

KEY POINT

When you chose a variable name, it should be descriptive of the data it will hold, e.g. distance or length. Long variable names are easier to misspell!

SUBJECT VOCABULARY

constant a 'container' that holds a value that never changes; like variables, constants have unique identifiers

GENERAL VOCABULARY

quotient a result found by dividing one quantity by another

modulus the remainder after the division of one number by another

power the small number written to the right and above another number to show how many times it should be multiplied by itself

Variables play an important role in algorithms and programming. The value stored by a variable can change as a program is running. Variables are extremely useful in programming because they make it possible for the same program to process different sets of data.

A **constant** is the opposite of a variable. It is a 'container' that holds a value that always stays the same. Constants are useful for storing fixed information, such as the value of pi, the number of litres in a gallon or the number of months in a year.

Each variable and constant in an algorithm has to have a unique identifier. It is important to choose descriptive names for identifiers. This will make your code much easier to read. For example, a variable to hold a user's first name could be given the identifier `firstName` to show the data it contains. If it were given the identifier `X` instead, it wouldn't clearly show what data it contained.

NAMING CONVENTIONS FOR VARIABLES AND CONSTANTS

It is sensible to write identifiers in the same way throughout an algorithm. A common method is to use 'camel case' for compound words (e.g. `firstName`, `secondName`) with no space between words and the second word starting with a capital letter. Alternatively, you could capitalise the first letter of both words, e.g. `FirstName, SecondName`, or separate the words with an underscore, e.g. `first_name, second_name`, known as 'snake case'.

SKILLS PROBLEM SOLVING, ANALYSIS

ACTIVITY 5

WRITING ALGORITHMS IN PSEUDOCODE

Here is a written description of an algorithm:

Enter the first number.
Enter the second number.
The third number is equal to the first number multiplied by the second number.
Display the third number.

Express this algorithm in pseudocode.

KEY POINT

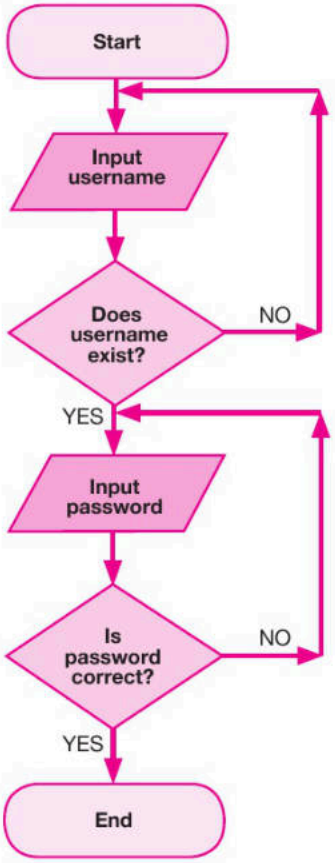
Whichever method you use for naming conventions, you must use it consistently and not keep switching between the different methods.

SKILLS PROBLEM SOLVING

ACTIVITY 6

WRITTEN DESCRIPTIONS OF ALGORITHMS

This algorithm is displayed as a flowchart.



▲ **Figure 1.5** Flowchart of an algorithm

Produce a written description of this algorithm.

EXTEND YOUR KNOWLEDGE

When you enter a search term into Google®, a list of links to websites is returned. But why are they presented in that particular order? With a partner, research the `PageRank` algorithm that Google uses to rate the importance of websites and write a short report about your findings.

CHECKPOINT

Strengthen

SKILLS CRITICAL THINKING

S1 Produce a written description of an algorithm for borrowing a book from the library.

SKILLS DECISION MAKING

S2 What is the function of each of the seven arithmetic operators?

SKILLS REASONING

S3 What is a variable? Why are they useful?

SKILLS CRITICAL THINKING

S4 What is the difference between a variable and a constant?

Challenge

SKILLS CRITICAL THINKING, PROBLEM SOLVING

C1 Produce a flowchart describing an algorithm for making a cheese sandwich.

SKILLS PROBLEM SOLVING

C2 Write an algorithm expressed in pseudocode that receives three numbers from the keyboard, then calculates and displays the average.

How confident do you feel about your answers to these questions? If you're not sure you answered them well, try the following activities again.

- For S1 re-read 'The relationship between algorithms and programs'.
- For S2 study Table 1.1.
- For S3 and S4 look again at 'Variables and constants'.

SUMMARY

- An algorithm is a precise method for solving a problem.
- Algorithms can be displayed as written descriptions, flowcharts and in pseudocode.
- Pseudocode is a **structured**, code-like language.
- Pseudocode is translated into program code.
- Arithmetic operators are used in calculations.
- Variables and constants are 'containers' for storing data. The value stored in a variable can change, whereas the value of a constant never changes.
- Selecting descriptive names for identifiers makes code easier to read.

GENERAL VOCABULARY

structured organised so that the parts work well together

2 CREATING ALGORITHMS

In this section you will learn how to create algorithms to solve particular problems using the constructs of sequence, selection and iteration. You will also practise displaying algorithms in flowcharts and pseudocode.

LEARNING OBJECTIVES

- Understand how to create an algorithm to solve a particular problem
- Make use of programming constructs (sequence, selection and iteration) and use appropriate conventions (flowchart, pseudocode, written description, draft program code)

ALGORITHMS FOR COMPUTERS

SUBJECT VOCABULARY

construct a smaller part from which something is built. Letters and numbers (i.e. a to z and 0 to 9) are the constructs we use to build our language and convey meaning. Bricks and cement are the basic constructs of a building

selection a construct that allows a choice to be made between different alternatives

iteration a construct that means a process is repeated. An action is repeated until a **condition** is met or a particular outcome is reached. It is often referred to as a 'loop'

GENERAL VOCABULARY

ambiguous when a statement or command does not have one obvious meaning but can be interpreted in different ways

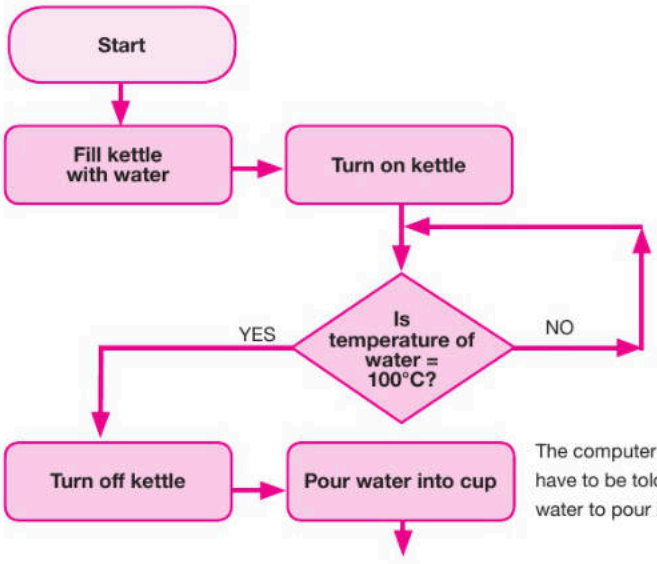
condition something that must happen before something else can happen

There was an **ambiguous** statement in the algorithm for making a cup of coffee. After filling the kettle with water and adding coffee to the cup, the next instruction was 'Wait for water to boil'.

A human can understand that this instruction means they have to keep checking the kettle over and over again until the water is boiling, but a computer is unable to understand an instruction like this in the same way. It would just wait. And wait. Forever.

Even worse, the algorithm didn't state clearly how to tell the water was boiling. Through experience, we humans assume the water is boiling when there is lots of steam, sound and bubbles; or, even better, when the kettle turns itself off. An algorithm for a computer would have to state that it must wait until the water reached 100°C.

A version of this part of the algorithm, suitable for a computer, is shown in Figure 1.6. This example introduces two new **constructs** from which algorithms are created. We have already met the construct sequence – step-by-step instructions in the correct order. To add to this, we now have **selection** and **iteration**.

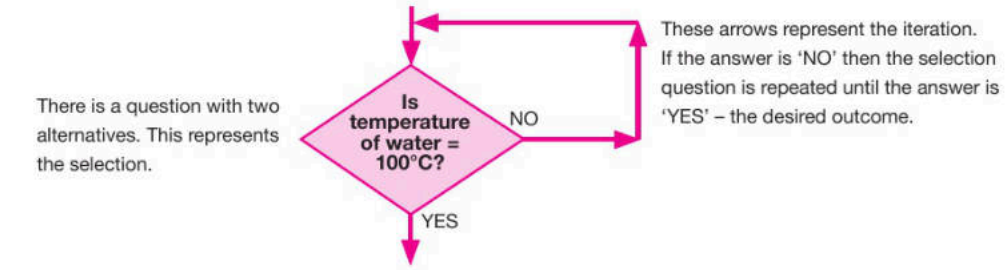


The computer would also have to be told how much water to pour into the cup!

▶ **Figure 1.6** Part of an algorithm suitable for a computer for making coffee

REPRESENTING SELECTION AND ITERATION IN A FLOWCHART

Selection and iteration are represented in a flowchart as shown in Figure 1.7.



▶ **Figure 1.7** Selection and iteration in a flowchart

DID YOU KNOW?

We use iteration in our daily lives whenever we carry out an action over and over again. For example, at mealtimes we keep on eating until our plate is empty or we have had enough to eat.

When we're travelling by car and the traffic lights are red, we have to keep waiting until they change to green.

An actor repeats their lines over and over again, learning more each time until they know them all.

SKILLS REASONING, PROBLEM SOLVING

ACTIVITY 7

GUESSING GAMES

A student is creating a guessing game. A player has to enter a number no greater than 10. If it is too high, they are informed that they have made an error. But if it is within the range 1 to 10, they are told whether or not they have guessed the correct number. (Assume that the correct number is 3.)

Can you make an algorithm to solve this problem and express it as a written description and a flowchart?

Compare your solution to others in your group.

Check that they are correct and would produce the correct outcome.

Are some of the algorithms more efficient than others? Do they use fewer commands?

SKILLS PROBLEM SOLVING, ANALYSIS

ACTIVITY 8

CALCULATING GRADES

A school uses this algorithm to calculate the grade that students achieve in end-of-topic tests.

```
RECEIVE testScore FROM KEYBOARD
IF testScore >= 80 THEN
    SEND 'A' TO DISPLAY
ELSE
    IF testScore >= 70 THEN
        SEND 'B' TO DISPLAY
    ELSE
        IF testScore >= 60 THEN
            SEND 'C' TO DISPLAY
        ELSE
            IF testScore > 0 THEN
                SEND 'D' TO DISPLAY
            ELSE
                SEND 'FAIL' TO DISPLAY
```

```
            END IF
        END IF
    END IF
END IF
```

What would be the output of this algorithm for these test scores: 91, 56 and 78?

ITERATION

When writing programs, it is often necessary to repeat the same set of statements several times. Instead of making multiple copies of the statements, you can use iteration to repeat them. The algorithm for making a cup of coffee includes an instruction to keep waiting until the water in the kettle boils.

CHECKPOINT

SKILLS CRITICAL THINKING, DECISION MAKING

Strengthen

S1 How are sequence, selection and iteration used in algorithms? Give examples to justify your answer.

SKILLS PROBLEM SOLVING

Challenge

C1 Develop an algorithm using a flowchart that asks the user to enter their height (in metres) and weight (in kilograms) and displays their body mass index (BMI). The formula for calculating BMI is weight/height^2.

SKILLS PROBLEM SOLVING

C2 Develop an algorithm expressed as a flowchart to control the heating in a house. A thermostat monitors the temperature within the house. During the week the temperature should be 20°C between 06.00 and 08.30 in the morning and between 17.30 and 22.00 at night. At weekends it should be 22°C between 08.00 and 23.00. If the temperature in the house falls below 10°C at any time the boiler is switched on.

How confident do you feel about your answers to these questions? If you're not sure you answered them well, try the following activities again.

- For S1 have a look at the Subject vocabulary sections in 'Understanding algorithms' and 'Creating algorithms'.

SUMMARY

- The constructs sequence, selection and iteration are the basic building blocks of algorithms.

4 DECOMPOSITION AND ABSTRACTION

When computer scientists attempt to solve problems by producing algorithms and coding them into programs, they approach the problems in a particular way. This method has been given the name 'computational thinking'. Before they create a structured solution or algorithm and code it into a program, they must define and analyse the problems. This section will introduce two of the **thought processes** they use – decomposition and abstraction.

GENERAL VOCABULARY

thought process the act of using your mind to consider or think about something

LEARNING OBJECTIVES

- Analyse a problem, investigate requirements (inputs, outputs, processing, initialisation) and design solutions
- Decompose a problem into smaller sub-problems
- Understand how abstraction can be used effectively to model aspects of the real world
- Program abstractions of real-world examples

PROBLEM SOLVING

SUBJECT VOCABULARY

computational thinking the thought processes involved in formulating problems and their solutions so that the solutions are represented in a form that can be effectively carried out by a computer

decomposition breaking a problem down into smaller, more manageable parts, which are then easier to solve

abstraction the process of removing or hiding unnecessary detail so that only the important points remain

The tasks of a computer scientist include defining and analysing problems; creating structured solutions – algorithms; and coding the solutions into a form that can be implemented by a computer. These tasks are part of what is known as **computational thinking**.

One of the skills required for computational thinking is algorithm design (which we've covered in detail in this unit). If there is a fault in the algorithm design, then the program will not work, however good a coder you are. Two other skills are **decomposition** and **abstraction**.

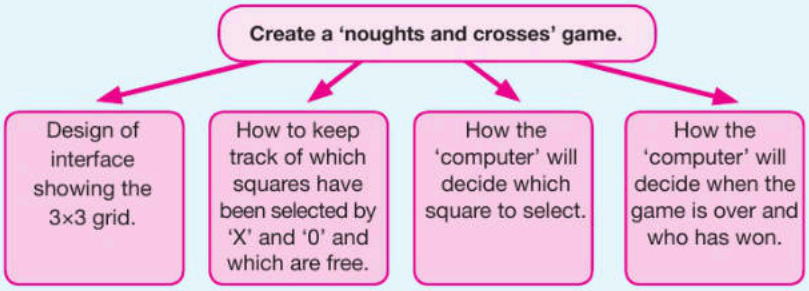
DECOMPOSITION

Decomposition is usually the first step in the problem-solving process. Once a problem has been broken down and the sub-problems have been identified, algorithms can be developed to solve each of them.

Decomposition means that sub-problems can be worked on by different teams at the same time. As smaller algorithms are developed for each sub-problem, it is easier to spot and correct errors. When the algorithm is developed into a program, code can be used again.

WORKED EXAMPLE

A student has been set the task of creating a computer version of the game 'noughts and crosses' (also known as 'tic-tac-toe') where a user plays against the computer.



▶ **Figure 1.12** Sub-problems to be solved to create a noughts and crosses computer program

The diagram shows some of the sub-problems that must be solved in order to solve the complete problem and create a version of the game.

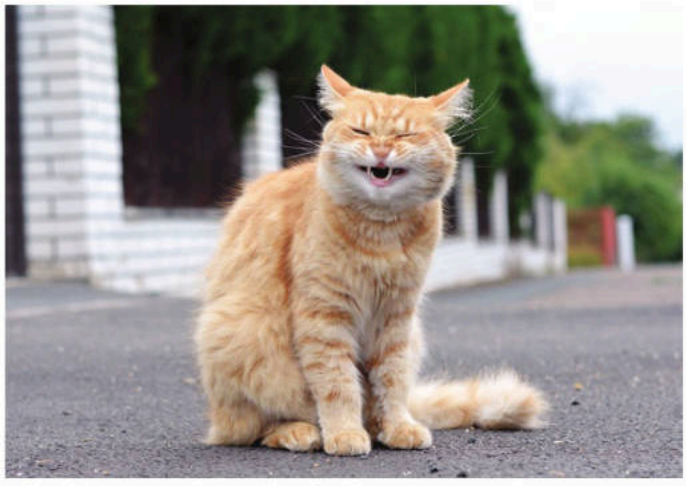
ABSTRACTION

We use abstraction all the time in our daily lives. We **abstract** the essential features of something so that we can understand what people are trying to communicate.

GENERAL VOCABULARY

abstract to remove something from somewhere

Somebody might say, 'I was walking down the street when I saw a cat'. You immediately understand what they mean by 'street' – probably a road with a pavement and houses or shops along the side of it. Similarly, you can picture the cat – a small animal with fur, four legs and a tail. An animal that is basically 'cattish'. You have extracted the basic properties of animals called cats so that you can recognise one when you see one, or imagine one when somebody talks about a cat.



▲ Is this the street and cat you imagined?

What you imagine is very unlikely to match the actual street and cat that the person experienced. But, because of our ability to abstract, the person did not

have to go into unnecessary detail about exactly where they were and what they saw. They wouldn't get very far with the story if they did.

When we create algorithms, we abstract the basic details of the problem and represent them in a way that a computer is able to process.

WORKED EXAMPLE

Yasmin is designing a computer version of a game in which users have to throw a die to find out their number of moves.

In the computer game, the users can't have an actual die, so she will have to design a 'pretend' or virtual die that behaves in exactly the same way as a real-life die.

Yasmin will have to use her powers of abstraction to work out the essential features of a die and then represent them in computer code.

To represent the die, she will have to create a routine that will select a random number from 1 to 6 because that's what a die does.

Yasmin has used abstraction to **model** a real-life event.

GENERAL VOCABULARY

model to make a simple version of something to show how it works

LEVELS OF ABSTRACTION

There are different levels or types of abstraction. The higher the level of abstraction, the less detail is required. We use abstraction all the time in accomplishing everyday tasks.

When programmers write the 'print' command they do not have to bother about all of the details of how this will be accomplished. They are removed from them. They are at a certain level of abstraction.

A driver turning the ignition key to start a car does not have to understand how the engine works or how the spark to ignite the petrol is generated. It just happens and they can simply drive the car. That is abstraction.



▲ A game of noughts and crosses

AN EXAMPLE – NOUGHTS AND CROSSES

Figure 1.12 showed some of the sub-problems that the problem of creating a noughts and crosses game could be divided into. The following could be written at a high level of abstraction.

- The computer goes first. Then the user. This continues until either one wins, or all of the squares have been used.

Immediately a pattern can be recognised – a loop will be needed.

Inputs and outputs

The following inputs from the user will be needed.

- Start the game.
- Entries for the user.
- Select a new game or finish.

The following outputs will be needed.

- A message to inform the user when it is their turn.
- A message to inform the user if they try to select a square that has already been used.

5 DEVELOP CODE

PROGRAMMING LANGUAGES

Throughout the book, we have coloured the different programming languages in order for you to easily find the one you are looking for.

The colours are:

```
Pearson Edexcel pseudocode
```

```
Python
```

```
Java
```

```
C#
```

Once an algorithm has been developed to solve a particular problem, it has to be coded into the programming language that the developer is using. This usually means that the written descriptions, flowcharts and pseudocode have to be converted into actual programming code.

As you will be expected to understand, use and edit Pearson Edexcel `pseudocode` in the examination, examples will be given in the pseudocode as well as in `Python`, `Java` and `C#`.

LEARNING OBJECTIVES

- Explain the difference between algorithms and programs
- Code an algorithm in pseudocode and a high-level programming language
- Describe the characteristics of data types and select appropriate data types for variables
- Use sequencing, selection and iteration constructs in your programs

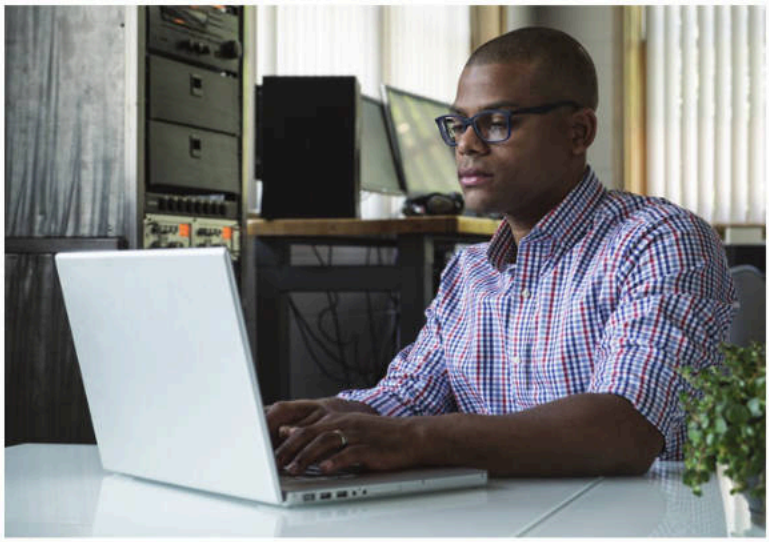
ALGORITHMS AND PROGRAMS

SUBJECT VOCABULARY

execution the process by which a computer carries out the instructions of a computer program

As you learnt in Unit 1, an algorithm is a precise method of solving a problem. It consists of a sequence of unambiguous, step-by-step instructions. A program is an algorithm that has been converted into program code so that it can be **executed** by a computer. A well-written algorithm should be free of logic errors and easy to code in any high-level language.

As part of this course you will learn to write programs in a high-level programming language. All high-level programming languages are like natural human languages, which makes them easier for humans to read and write but impossible for computers to understand without the help of a translator. You will learn more about how a program written in a high-level language is translated into machine code – the language of computers – in Unit 4. The aim of this unit is to develop your programming skills.



▶ A computer programmer at work writing code

DATA TYPES

GENERAL VOCABULARY

determine to discover facts about something

As you learnt in Unit 1, algorithms use variables (named memory locations) to store values. Variables have a variety of uses. For example, controlling the number of times a loop is executed, **determining** which branch of an IF statement is taken, keeping running totals and holding user input.

When algorithms are converted into programs, the computer needs to be told what type of data is stored in each variable. Every programming language has a number of built-in data types.

DATA TYPE	DESCRIPTION	EXAMPLE	EXAMPLES OF USE
integer	Used to store whole numbers without a fractional part	30	age = 30 number = 5
real or float	Used to store numbers with a fractional part (decimal place). Real numbers are sometimes referred to as floats (short for floating point)	25.5	weight = 25.5 price = 12.55
Boolean	Only has two possible values: True or False	False	correct = False lightOn = True
character*	A character can be a single letter, a symbol, a number or even a space. It is one of the four basic data types	'm'	gender = 'm' char = ':'
string	A set of characters which can include spaces and numbers and are treated as text rather than numbers	'the computer'	name = 'Catherine' type = 'liquid'

*Python does not have a character data type.

▲ **Table 2.1** Common data types

GENERAL VOCABULARY

assign to give somebody a particular task

When writing pseudocode, you don't have to specify the data types of variables. However, data types become much more important once you start programming in a high-level language. This is because the data type of a variable determines the operations that can be performed on it.

For example, the result of multiplying a value by 5 differs according to its data type.

integer	8 * 5 = 40
real	8.0 * 5 = 40.0
character	'8' * 5 = '88888'

HINT

Although you don't have to declare the data types of variables in pseudocode, it is a good idea to do so. You can simply list the variables and their data types at the start of an algorithm, for example:

```
INTEGER age
REAL weight
BOOLEAN correct
CHARACTER gender
```

You can also specify the data type of a variable in the RECEIVE statement, for example:

```
RECEIVE age FROM (INTEGER) KEYBOARD
RECEIVE price FROM (REAL) KEYBOARD
```

The method of declaring variables differs between programming languages. Some languages, such as Python, automatically select the appropriate data type for a variable based on the data **assigned** to it. Others, such as Java and C#, require the data type of variables to be declared before the variables can be used.

VARIABLE INITIALISATION

When a variable is declared, the computer gives it a location in its memory. Initially, this location is empty, so before a variable can be used it has to be given a value.

SUBJECT VOCABULARY

initialise to set variables to their starting values at the beginning of a program or subprogram

initialisation the process of assigning an initial value to a variable

assignment statement the SET…TO command is used to initialise variables in pseudocode, for example:

```
SET anotherGo TO 0
SET correct TO False
```

You can put an initial value into a variable by:

- initialising it when the program is run (e.g. `SET total TO 0`, in pseudocode)
- reading a value from a keyboard or other device (e.g. `RECEIVE admissionCharge FROM (INTEGER) KEYBOARD`).

Once a variable has been **initialised** an **assignment statement** is used to change its value (e.g. `SET total TO total + admissionCharge`).

In Python this would be:

```
total = 0
total = total + admissionCharge
```

In Java this would be:

```
Scanner scan = new Scanner(System.in);
int admissionCharge = scan.nextInt();
int total = 0;
total = total + admissionCharge;
```

In C#, variables must be declared before use. When you declare a variable, you need to state the data type that the variable will store, for example:

```
// declare a variable called total that will be used to store floating point numbers and assign the value 0.0 to it
float total = 0.0;
// add the value stored in the variable admissionsCharge to the total
total = total + admissionsCharge;
```

GENERAL VOCABULARY

intended designed for a certain purpose

If a variable, such as a loop counter, is **intended** to hold a running total, then it should always be initialised to a starting value. Some programming languages won't execute if the programmer fails to do this; others will do so but may well produce some unexpected results.

SKILLS CRITICAL THINKING, DECISION MAKING

ACTIVITY 1

DATA TYPES

1 Investigate what data types are available in the high-level language you are studying. Produce a table similar to Table 2.1 to give a summary of your findings.

2 What do you think is an appropriate data type for each of these items?
   a the test score of an individual learner
   b the average score for a group of learners
   c whether or not the pass mark for the test has been achieved.

3 Look back over the algorithms you wrote in Unit 1 and find instances of variable initialisation.

SUBJECT VOCABULARY

type coercion the process of converting the value stored in a variable from one data type to another

TYPE COERCION

Sometimes the data type of a variable gets changed during program execution. This is known as **type coercion**. For example, if an integer value and a real value are used in an arithmetic operation, the result will always be a real.

In Python, type coercion is done automatically, for example, the output from the following program is 3.25.

```
x = 1
y = 2.25
z = x + y
print(z)
```

Type coercion is also automatic in Java:

```
int x = 1;
double y = 2.25;
double z = x + y;
System.out.print(z);
```

In C# this is generally referred to as casting. As with Python this can be automatic for many data type conversions and is known as implicit casting. For more complicated conversions you may need to research explicit casting.

One frequently used type of explicit casting is converting from a string to a numeric value. This is used when getting input from a user as `Console.Readline`. For example, the following C# code will output 3.25 and demonstrates automatic type coercion.

```
int x;                  // declares a variable called x that will store integers
double y, z;            // declares two variables called y and z that will store
                        floating point numbers
x = 1;
y = 2.25;
z = x + y;
Console.WriteLine(z);
```

SKILLS CRITICAL THINKING, PROBLEM SOLVING, DECISION MAKING

ACTIVITY 2

MONITORING VISITOR NUMBERS



A theme park uses a program to monitor the number of people entering and exiting the park. The maximum number of visitors at any one time must not exceed 10 000. When the number of people in the park reaches the maximum, a 'Park Full' message is displayed at the entrance gate. Children can visit the park free of charge. Adults must pay AED 125 admission. The program records the amount of money collected at the gate.

1 What are the variables needed in the program?

2 Select an appropriate data type for each variable and constant.

COMMAND SEQUENCE, SELECTION AND ITERATION

In Unit 1 you learnt that the three key building blocks of algorithms are command sequence, selection and iteration. In this unit, you will have the opportunity to implement these constructs in the high-level programming language you are studying.

SKILLS CRITICAL THINKING, PROBLEM SOLVING, CREATIVITY

ACTIVITY 3

UNDERSTANDING ALGORITHMS

Read the following algorithm written in pseudocode and then answer the questions below.

```
RECEIVE number1 FROM (INTEGER) KEYBOARD
RECEIVE number2 FROM (INTEGER) KEYBOARD
SET result1 TO number1 / number2
SEND result1 TO DISPLAY
SET result2 TO number1 MOD number2
SEND result2 TO DISPLAY
SET result3 TO number1 DIV number2
SEND result3 TO DISPLAY
```

1 What does this algorithm do?

2 What is the output of the algorithm, given the following inputs:
   a 4, 2
   b 10, 3
   c 20, 6?

Implement this algorithm in the high-level language you are studying.

SELECTION

The selection construct is used to create a branch in a program. The computer selects which branch to follow based on the outcome of a condition, using an `IF...THEN...ELSE` statement. For example, in pseudocode:

```
IF day = 'Saturday' OR day = 'Sunday' THEN
  SET alarm TO 11
ELSE
  SET alarm TO 8
END IF
```

SUBJECT VOCABULARY

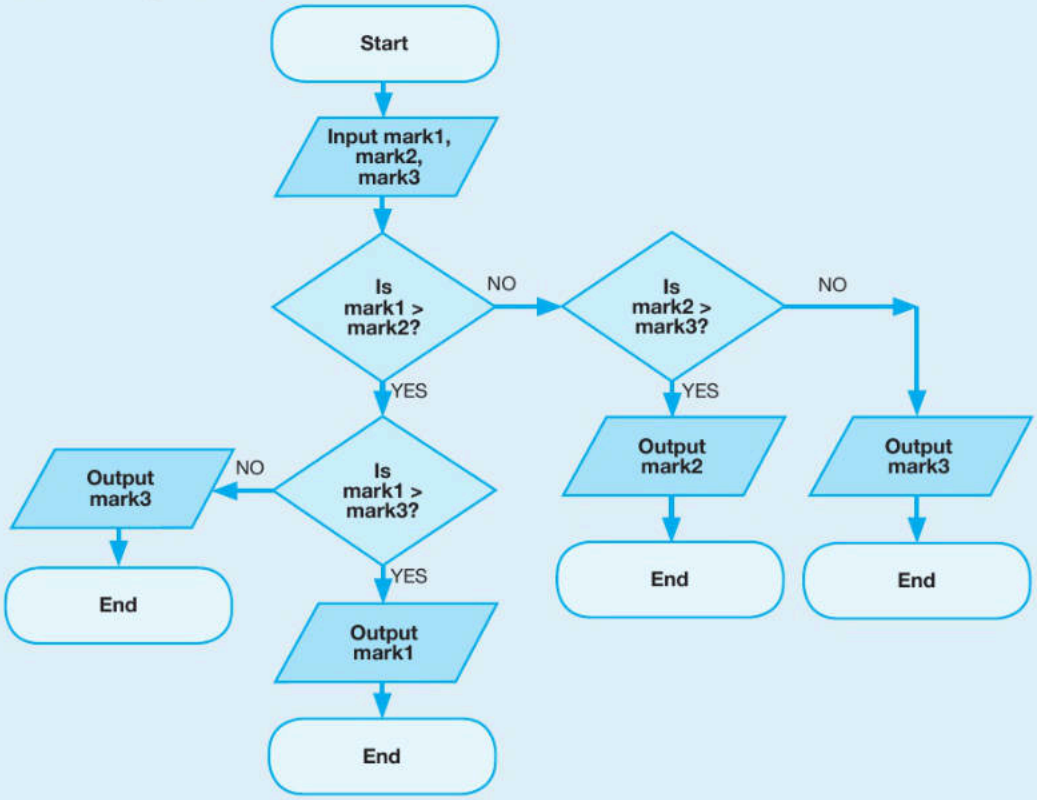
nested IF statement a nested IF statement consists of one or more IF statements placed inside each other. A nested IF is used where there are more than two possible courses of action

A standard `IF...THEN...ELSE` statement provides two alternatives. If there are more than two, then in Pearson Edexcel pseudocode a nested `IF` must be used. However, many high-level programming languages have an additional built-in selection construct that does away with the need for a **nested IF statement**.

WORKED EXAMPLE

A learner handed in three homework assignments, which were each given a mark out of 10. All the marks were different. Write an algorithm that would print out the highest mark.

Figure 2.1 shows the algorithm expressed as a flowchart.



▲ **Figure 2.1** Flowchart of an algorithm to print out the highest homework mark

HINT

When you are creating nested `IF` statements, you have to ensure that each one is completed with an `END IF` statement at the correct indentation level. Some programming languages do not need an `END IF` statement and just use the indentation levels to indicate when statements are grouped.

In Pearson Edexcel pseudocode, this algorithm could be expressed as:

```
RECEIVE mark1 FROM KEYBOARD
RECEIVE mark2 FROM KEYBOARD
RECEIVE mark3 FROM KEYBOARD
IF mark1 > mark2 AND mark1 > mark3 THEN
    SEND mark1 TO DISPLAY
ELSE
    IF mark2 > mark1 AND mark2 > mark3 THEN
        SEND mark2 TO DISPLAY
    ELSE
        IF mark3 > mark1 AND mark3 > mark2 THEN
            Send mark3 TO DISPLAY
        END IF
    END IF
END IF
```

In Python, Java and C# this does not have to be done as they have an 'else if' statement.

In Python the 'else if' statement is `elif` and the algorithm above could be:

```
mark1 = input('Please enter the first mark')
mark2 = input('Please enter the second mark')
mark3 = input('Please enter the third mark')

if mark1 > mark2 and mark1 > mark3:
    print(mark1)
elif mark2 > mark1 and mark2 > mark3:
    print(mark2)
elif mark3 > mark1 and mark3 > mark2:
    print(mark3)
```

Here is the algorithm in Java:

```
import java.util.*;
class Main {
  public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
    System.out.print("Please enter the first mark: ");
    int mark1 = scanner.nextInt();
    System.out.print("Please enter the second mark: ");
    int mark2 = scanner.nextInt();
    System.out.print("Please enter the third mark: ");
    int mark3 = scanner.nextInt();
    scanner.close();

    if (mark1 > mark2 && mark1 > mark3) {
      System.out.print(mark1);
    } else if (mark2 > mark1 && mark2 > mark3) {
      System.out.print(mark2);
    } else if (mark3 > mark1 && mark3 > mark2) {
      System.out.print(mark3);
    }
  }
}
```

And in C#:

```
string mark1String, mark2String, mark3String;
int mark1, mark2, mark3;

Console.WriteLine("Please enter the first mark");
mark1String = Console.ReadLine();    // read input into string
mark1 = int.Parse(mark1String);      // convert string to integer

Console.WriteLine("Please enter the second mark");
mark2String = Console.ReadLine();    // read input into string
mark2 = int.Parse(mark2String);      // convert string to integer
```

```
Console.WriteLine("Please enter the third mark");
mark3String = Console.ReadLine();    // read input into string
mark3 = int.Parse(mark3String);      // convert string to integer

if (mark1 > mark2 && mark1 > mark3)
{
    Console.WriteLine(mark1);
}
else if (mark2 > mark1  && mark2 > mark3)
{
   Console.WriteLine(mark2);
}
else if (mark3 > mark1 && mark3 > mark2)
{
   Console.WriteLine(mark3);
}
```

These examples use **relational** and **logical operators**.

SUBJECT VOCABULARY

relational operator an operator that tests the relationship between two entities

logical operator a Boolean operator using AND, OR and NOT

RELATIONAL OPERATORS

The relational operators are used to compare two values and in Python, Java and C# are all the same.

RELATIONAL OPERATOR	PYTHON, JAVA, C#
Equal to	==
Greater than	>
Greater than or equal to	>=
Less than	<
Less than or equal to	<=
Not equal to	!=

ACTIVITY 4

Look at the following algorithm and answer the questions.

```
IF score <= highScore THEN
    SEND 'You haven't beaten your high score.' TO DISPLAY
ELSE
    SEND 'You've exceeded your high score!' TO DISPLAY
END IF
```

What is the output of the algorithm when

- `score` = 5 and `highScore` = 10?
- `score` = 20 and `highScore` = 10?
- `score` = 15 and `highScore` = 15?

LOGICAL OPERATORS

AND

If two conditions are joined by the `AND` operator, then they must both be true for the whole statement to be true.

OR

If two conditions are joined by the `OR` operator, then either one must be true for the whole statement to be true.

NOT

The `NOT` operator reverses the logic of the `AND` and `OR` statements. The statement `IF A = 3 AND B = 6` will be true only if the conditions are met, i.e. A and B are both equal to the values stated.

The statement `IF NOT (A = 3 AND B = 6)` will be true whenever both A and B are NOT equal to the values stated, i.e. either or both are not equal to those values.

Logical operators in high-level languages.

LOGICAL OPERATOR	PYTHON	JAVA	C#
AND	and	&	&&
OR	or	|	||
NOT	not	!	!

ACTIVITY 5

A driving school uses this rule to estimate how many lessons a learner will require.

- Every learner requires at least 20 lessons.
- Learners over the age of 18 require more lessons (two additional lessons for each year over 18).

Create a program in a high-level language that inputs a learner's age and calculates the number of driving lessons they will need.

LOOPS

A loop is another name for an iteration. Loops are used to make a computer repeat a set of instructions more than once. There are two types of loop: definite and indefinite.

A definite loop is used when you know in advance how often the instructions in the body of the loop are to be repeated. For example, if you want the computer to display a character on the screen for a fixed amount of time and then remove it.

An indefinite loop is used when the number of times a loop will need to be repeated is not known in advance. For example, if you want to give a user the option of playing a game as often as they want. Indefinite loops are repeated until a specified condition is reached.

Every programming language has a number of built-in loop constructs. You will need to explore the ones provided in the language you are studying.

6 MAKING PROGRAMS EASY TO READ

Developers usually work in teams and it is important that they understand how each other's code and programs work, especially if there are errors that need correcting. Code should be written in standard ways and be explained using comments.

LEARNING OBJECTIVES

- Explain the benefit of producing programs that are easy to read
- Use techniques to improve the readability of code and describe how code works

CODE READABILITY

You should always try to ensure that any code you write is easy to read and understand. We refer to this as 'readability'. This benefits you and anyone else who needs to understand how your programs work.

It is surprising how quickly you forget. Try revisiting the programs you have already written in this unit and make sure it is still clear to you what they do and how they work. Imagine how much more difficult it would be to make sense of a complex program with lots of variables, subprograms, nested loops and multiple selection statements.

Programming is not a solo activity. Programmers usually work in teams, with each programmer developing a different part of the program. This only works if they all adopt a standard approach to writing readable code.

▶ **Figure 2.2** An example of Python code

Python 3.4.1: example2.py - C:/Python/example2.py

File Edit Format Run Options Windows Help

```
import random
game=True
while game==True:
    Num1=int(input("Choose one of the following: 1 Rock, 2 Paper, or 3 Scissors:"))
    Num2=random.randint(1,3)
    if Num2==1:
        print("Computer chooses Rock")
    elif Num2==2:
        print("Computer chooses Paper")
    else:
        print("Computer chooses Scissors")
    if Num1==Num2:
        print("It's a draw")
    elif Num1==1 and Num2==3:
        print("You win!")
    elif Num1==2 and Num2==1:
        print("You win!")
    elif Num1==3 and Num2==2:
        print("You win!")
    else:
        print("Computer wins")
    text=input("Play again? Y/N:")
    if text!="Y":
        game=False
```

Ln: 26 Col: 0

GENERAL VOCABULARY

practice the way that you do something

SUBJECT VOCABULARY

block of code a grouping of two or more code statements

The programmer who produced the program in Figure 2.2 did not follow good **practice**. There are a number of ways that the readability of this code could be improved.

- Use descriptive names for variables (e.g. `userChoice` instead of `Num1`).
- Add blank lines between different **blocks of code** to make them stand out.
- Add comments that explain what each part of the code does.

HINT

Here's how to write comments in Python, Java and C#:

- **Python:** start each line of a comment with the # symbol
- **Java:** start single line comment with // or start a multi-line comment with /* and end it with */
- **C#:** Same as for Java.



▲ Programmers often work in teams; it is vital that any coding they share is clear and error free

Table 2.5 lists the techniques you should use to make your programs easy to read and understand.

DID YOU KNOW?

Microsoft Windows® uses roughly 50 million lines of code – code readability obviously becomes more and more important as programs get larger.

TECHNIQUE	DESCRIPTION
Comments	Comments should be used to explain what each part of the program does.
Descriptive names	Using descriptive identifiers for variables, constants and subprograms helps to make their purpose clear.
Indentation	Indentation makes it easier to see where each block of code starts and finishes. Getting the indentation wrong in Python will result in the program not running or not producing the expected outcomes.
White space	Adding blank lines between different blocks of code makes them stand out.

▲ **Table 2.5** Techniques for program clarity

SKILLS CRITICAL THINKING, PROBLEM SOLVING, CREATIVITY

ACTIVITY 11

REWRITING AND IMPLEMENTING ALGORITHMS

1 Program the following algorithm in a high-level language and make it readable by adding comments and indentation.

```
SET x TO 10
WHILE x >= 0 DO
  IF x > 0
    SEND x TO DISPLAY
  ELSE
    SEND 'Blast Off' TO DISPLAY
    END IF
  SET x TO x -1
END WHILE
```



▲ Calculators have a number of useful functions

SKILLS REASONING

SKILLS CRITICAL THINKING

SKILLS PROBLEM SOLVING

2 Develop an algorithm for a simple calculator and code it in a high-level language that:
   - **a** allows the user to choose from these options: addition, subtraction, division and multiplication
   - **b** prompts the user to input two numbers
   - **c** performs the calculation and displays the result
   - **d** offers the user the option of performing another calculation.

CHECKPOINT

Strengthen

S1 Why it is important to make your code easy to read?

S2 Outline the four techniques that a programmer should use to make code easy to read.

Challenge

C1 Revisit the programs you have already written. Do you still understand what they do and how they work? If not, try to improve their readability.

KEY POINT

Readability of code is important. You should always make your code easy to read.

How confident do you feel about your answers to these questions? If you're not sure you answered them well, look again at Table 2.5.

SUMMARY

- Using comments, descriptive names, indentation and white space makes code easier to read.
- Producing readable code makes it easier to understand what a program does and how it does it.

9 INPUT/OUTPUT

All program data has to be entered (input) and information is output. The input could be automatic, e.g. from a sensor, but is often provided by human users. It is essential that this input is checked to ensure that it is what could reasonably be expected and falls within a certain range. Incorrect input could cause problems or even catastrophes.

LEARNING OBJECTIVES

- Explain the need for validation
- Write code that validates user input
- Write code that accepts and responds appropriately to user input
- Write code that reads to and writes from a text file

USER INPUT

Most programs require some form of input either from a user or from a file. You already know how to receive user input from a keyboard.

A program can be made much more 'user friendly' by displaying helpful messages informing users of what they are expected to enter and confirming successful input.

VALIDATION

It is important to ensure that data entered by the user is valid, as invalid data can cause a program to behave unexpectedly or even stop altogether. If the data entered into a program is incorrect, the output it produces will also be wrong. This is sometimes called the Garbage In, Garbage Out (GIGO) principle.

SUBJECT VOCABULARY

validation to check that the data entered by a user or from a file meets specified requirements

Any program that requires data entry should have appropriate forms of **validation** built in. But validation can't guarantee that the data entered is correct. It can only make sure that it is reasonable.

There are a number of different types of validation.

RANGE CHECK

A range check is used to ensure that the data entered is within a specified range. Study this algorithm, written in Pearson Edexcel pseudocode which checks that the number entered is between 1 and 10.

```
BOOLEAN valid
SET validNum TO False
WHILE validNum = False DO
   SEND 'Please enter a number between 1 and 10:' TO
   DISPLAY
   RECEIVE number FROM (INTEGER) KEYBOARD
   IF number >= 1 AND number <= 10 THEN
      SET validNum TO True
   END IF
END WHILE
SEND 'You have entered:' & number TO DISPLAY
```

The algorithm uses a Boolean variable named `validNum` as a status flag. It is initially set to False. The `WHILE` loop continues running until `validNum` is equal to True. An `IF` statement determines if the value of `validNum` should be set to True.

SKILLS CRITICAL THINKING, PROBLEM SOLVING

ACTIVITY 20

IMPLEMENTING RANGE CHECKS

Implement the range check algorithm in the high-level programming language you are studying.



PRESENCE CHECK

Another type of validation is a presence check. This simply ensures that a value has been entered, preventing the user from leaving an input blank.

This algorithm asks the user to input their name and uses a presence check to ensure they have entered a value. Any value will cause the loop to finish. It will keep asking the user to input their name until they input a value.

```
SET userName TO ' '
WHILE userName = ' ' DO
    RECEIVE userName FROM (STRING) KEYBOARD
END WHILE
SEND 'Hello' & userName TO DISPLAY
```

SKILLS CRITICAL THINKING, PROBLEM SOLVING

ACTIVITY 21

IMPLEMENTING A PRESENCE CHECK

Implement the presence check algorithm in the high-level programming language you are studying.

GENERAL VOCABULARY

predefined already defined before the start of something

LOOK-UP CHECK

A look-up check is used to test that a value is one of a **predefined** set of acceptable values. The list of acceptable values can be stored in a one-dimensional array.

This algorithm stores a list of valid form names in an array. It compares the form name entered by the user with the values in the array.

```
SET arrayForms TO ['7AXB', '7PDB', '7ARL', '7JEH']
RECEIVE form FROM (STRING) KEYBOARD
SET valid TO False
SET index TO 0
SET length TO LENGTH(arrayForms)
WHILE valid = False AND index < length DO
    IF form = arrayForms[index] THEN
        SET valid TO True
    END IF
    SET index TO index + 1
END WHILE
```

```
IF valid = True THEN
    SEND 'Valid form' TO DISPLAY
ELSE
    SEND 'The form you entered does not exist.' TO DISPLAY
END IF
```

SKILLS CRITICAL THINKING, PROBLEM SOLVING

ACTIVITY 22

IMPLEMENTING A LOOK-UP CHECK

Implement the look-up check algorithm in the high-level programming language you are studying.

LENGTH CHECK

It is sometimes necessary to check that the length of a value entered falls within a specified range. For example, all UK postcodes are between six and eight characters long, so validation could be used to check that the length of a postcode entered is within this range. Needless to say, that doesn't necessarily mean it is correct.

```
RECEIVE postCode FROM (STRING) KEYBOARD
SET length TO LENGTH(postCode)
IF length >= 6 AND length <= 8 THEN
    SEND 'Valid' TO DISPLAY
ELSE
    SEND 'Invalid' TO DISPLAY
END IF
```

SKILLS CRITICAL THINKING

ACTIVITY 23

IMPLEMENTING A LENGTH CHECK

Implement the length check algorithm in the high-level programming language you are studying.

TESTING VALIDATION RULES

It is important to test your validation rules to ensure they work as expected. You should use:

Normal data – This is data that is within the limits of what should be accepted by the program. For example, a password with seven characters fulfils the validation rule that states that passwords must be between six and eight characters in length.

Boundary data – This is data that is at the outer limits of what should be accepted by the program. For example, if a validation rule specifies that the range of acceptable values is >= 75 AND <= 100, then a value of 100 is at the upper limit and a value of 75 at the lower.

SUBJECT VOCABULARY

infinite loop a loop that is never-ending since the condition required to **terminate** the loop is never reached

GENERAL VOCABULARY

terminate to cause something to end or stop

example of an '**infinite loop**'. The algorithm should have been written as shown below.

```
SET index TO 1
WHILE index < 10
      SEND index TO DISPLAY
      SET index TO index + 1
END WHILE
```

SKILLS REASONING, PROBLEM SOLVING

ACTIVITY 29

FINDING AND CORRECTING ERRORS

Find and correct the errors in these algorithms.

Example 1

```
SET index TO 1
WHILE index < 10
    SEND index TO DISPLAY
END WHILE
```

Example 2

```
SET index TO 1
WHILE index < 10
    SEND index TO DISPLAY
    SET index TO index - 1
END WHILE
```

Example 3

```
SET index TO 1
WHILE index < 1
    SEND index TO DISPLAY
    SET index TO index + 1
END WHILE
```

TRACE TABLES

The formal way of checking the logic of an algorithm is to use a **trace table**.

SUBJECT VOCABULARY

logic error an error in an algorithm that results in incorrect or unexpected behaviour

trace table a technique used to identify any logic errors in algorithms. Each column represents a variable or output and each row a value of that variable

WORKED EXAMPLE

Create a trace table for the following algorithm written in Pearson Edexcel pseudocode.

```
SET number TO 3
FOR index FROM 1 TO 5 DO
    SET number1 TO number * index
    SET number2 TO number1 * 2
    IF number2 > 20 THEN
        SEND number2 TO DISPLAY
    END IF
END FOR
```

This algorithm can be traced as in Table 2.12.

NUMBER	INDEX	NUMBER1	NUMBER2	OUTPUT
3				
3	1	3	6	
3	2	6	12	
3	3	9	18	
3	4	12	24	24
3	5	15	30	30

▲ **Table 2.12** Trace table

The value of the variable number remains at 3 throughout, but as the index increases from 1 to 5, then so do the values of `numberl` and `number2`.

When the value of `number2` is greater than 20, its value is output.

WORKED EXAMPLE

Here is a program.

Python

```
y = 2
for x in range(1, 7):
      y = y + x
print(y)
```

Java

```
class Main {
  public static void main(String[] args) {
    int y = 2;
    for(int x = 1; x < 7; x++) {
      y += x;
    }
    System.out.println(y);
  }
}
```